Reflected DOM XSS

Lab Overview

- **Vulnerability:** Reflected DOM-based Cross-Site Scripting (XSS)
- **Category:** Client-Side Injection
- **Platform:** PortSwigger Web Security Academy
- **Severity:** Medium to High (context-dependent)
- **Status:** Successfully Exploited (after completing the lab)

Description

Reflected DOM XSS occurs when an application takes attacker-controlled input from an HTTP request and processes it in client-side JavaScript in an unsafe manner. If the application passes this input to a dangerous DOM sink without appropriate validation or encoding, injected JavaScript may execute in the victim's browser.

Unlike traditional reflected XSS, the vulnerable operation occurs within client-side JavaScript, even though the malicious input is delivered through a request.

Vulnerability Analysis

The application reflects user-controlled input into a response that is subsequently processed by JavaScript. The client-side code fails to handle the input safely, allowing an attacker to construct a URL that triggers JavaScript execution when opened by another user.

Proof of Concept (PoC)

1. Open the assigned PortSwigger lab.
2. Identify an input parameter reflected in the response.
3. Inspect the page's JavaScript to determine how the input is processed.
4. Trace the data to the unsafe DOM operation.
5. Construct a lab-specific test payload that demonstrates JavaScript execution.
6. Verify successful execution within the authorized lab environment.

Example test payload:

<img src=x onerror=alert(1)>

The payload is illustrative; the exact payload and injection context depend on the lab's vulnerable JavaScript and DOM sink.

Impact

Successful exploitation may allow an attacker to:

- Execute JavaScript in another user's browser under the vulnerable site's origin.
- Modify page content or perform unauthorized actions available to the victim.
- Access non-HttpOnly data exposed to page scripts.
- Conduct phishing or other client-side attacks.

The actual impact depends on the application's security controls and the victim's privileges.

Remediation

- Avoid unsafe DOM sinks such as `innerHTML` when inserting untrusted data.
- Prefer `textContent` or other safe DOM APIs.
- Validate input according to its expected format.
- Apply context-appropriate output encoding.
- Use a strict Content Security Policy (CSP) as an additional defense.
- Review client-side data flows from URL parameters to DOM manipulation.

References

- PortSwigger: DOM-based vulnerabilities
- OWASP: Cross-Site Scripting Prevention Cheat Sheet

Disclaimer

This report documents an authorized educational lab. Testing should only be performed on systems for which explicit permission has been granted.